CS 331 Operating Systems — Homework 8: Files

Name: Samvel Ghazaryan
Course: CS 331 Operating Systems (Sections A/B), Fall 2026
GitHub: https://github.com/samvelgh/OShomework/tree/main/hw8

All seven programs were compiled with `gcc -Wall -Wextra` on Ubuntu 24.04 (gcc 13.3.0), and none of them gave any warnings. I used only the POSIX calls `open`, `read`, `write`, `lseek`, `ftruncate` and `close` for the file work, and I check every call for errors and print the reason to stderr. Under each task is a screenshot of the run on my server.

Task 1: Copying a file

For the first task I wrote a small copy program. It asks for a source path and a destination path, reads both with `fgets()`, and strips the newline. Then I open the source with `open(src, O_RDONLY)` and the destination with `O_WRONLY | O_CREAT | O_TRUNC` and mode `0644`, so the file gets created if it doesn't exist and cleared if it does.

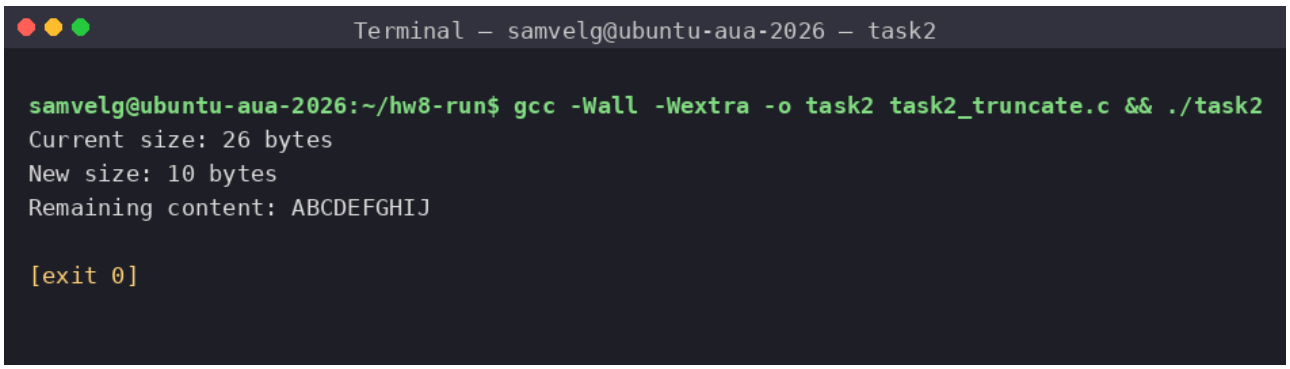
The actual copying is a loop that calls `read()` into a 4096-byte buffer until it returns 0. I put an inner loop around `write()` because a write can be partial, so I keep writing until the whole chunk is out. Every error goes to stderr with `strerror(errno)`. When I ran it on a three-line `source.txt` it printed "Copied 29 bytes", and `cat dest.txt` showed the same three lines.



Task 2: Truncating a file

Here the program creates `data.txt` and writes the alphabet A–Z into it, which is 26 bytes. After closing it I reopen it with `O_RDWR` and get the size with `lseek(fd, 0, SEEK_END)`, since the return value of that call is the offset of the end of the file.

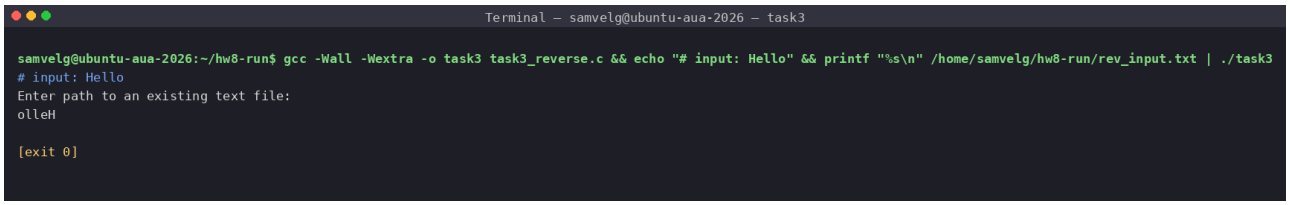
Then I call `ftruncate(fd, 10)` and check the size again the same way. To print what was left I rewind with `lseek(fd, 0, SEEK_SET)` and `read()` it back. The output showed the size going from 26 to 10 bytes, and the remaining content was `ABCDEFGHIJ`, which is what I expected.



Task 3: Reading a file backwards

For this one I open the file read-only and find its size with `lseek(fd, 0, SEEK_END)`. Then I loop from position `size - 1` down to 0. At each position I `lseek()` there with `SEEK_SET`, read one byte, and write it straight to stdout with `write(STDOUT_FILENO, &c, 1)`. At the end I print an extra newline so the prompt doesn't end up on the same line.

Reading one byte at a time isn't fast, but for this task it keeps the logic simple. My test file contained "Hello" followed by a newline, so the output starts with that newline and then shows `olleH`.



Task 4: Appending to a log with O_APPEND

This program always opens `log.txt` with `O_WRONLY | O_CREAT | O_APPEND`. I read one line from standard input with `read(0, ...)` instead of `fgets()`, drop the trailing newline if there is one, and build the line as `PID=<pid>:` plus the text plus my own newline, using `getpid()` for the PID. After writing it I print `lseek(fd, 0, SEEK_CUR)`.

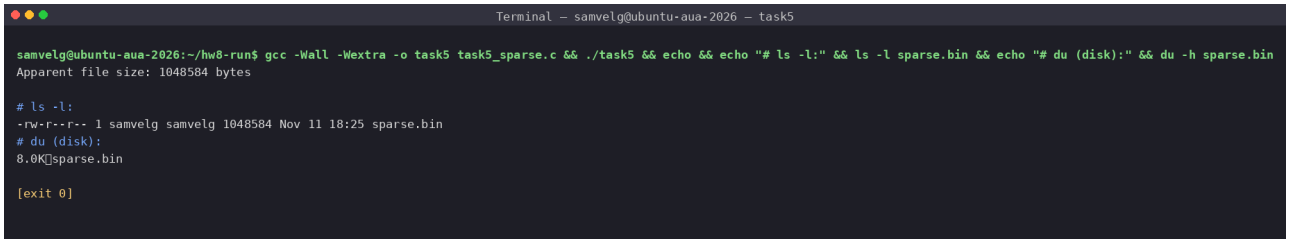
I ran it twice from the shell. The offsets came out as 31 and then 63, and the log had both lines with two different PIDs. The interesting part is why `SEEK_CUR` keeps growing even though I never seek: with `O_APPEND` the kernel moves every write to the current end of the file and leaves the offset there, so `SEEK_CUR` ends up reporting the end of the file.



Task 5: Creating a sparse file

Here I create `sparse.bin`, write `START` (5 bytes), and then call `lseek(fd, 1024*1024, SEEK_CUR)` to jump forward 1 MiB without writing anything. Then I write `END` (3 bytes). After reopening the file I get the size with `lseek(..., SEEK_END)`, and it printed 1048584 bytes, which is 5 + 1048576 + 3.

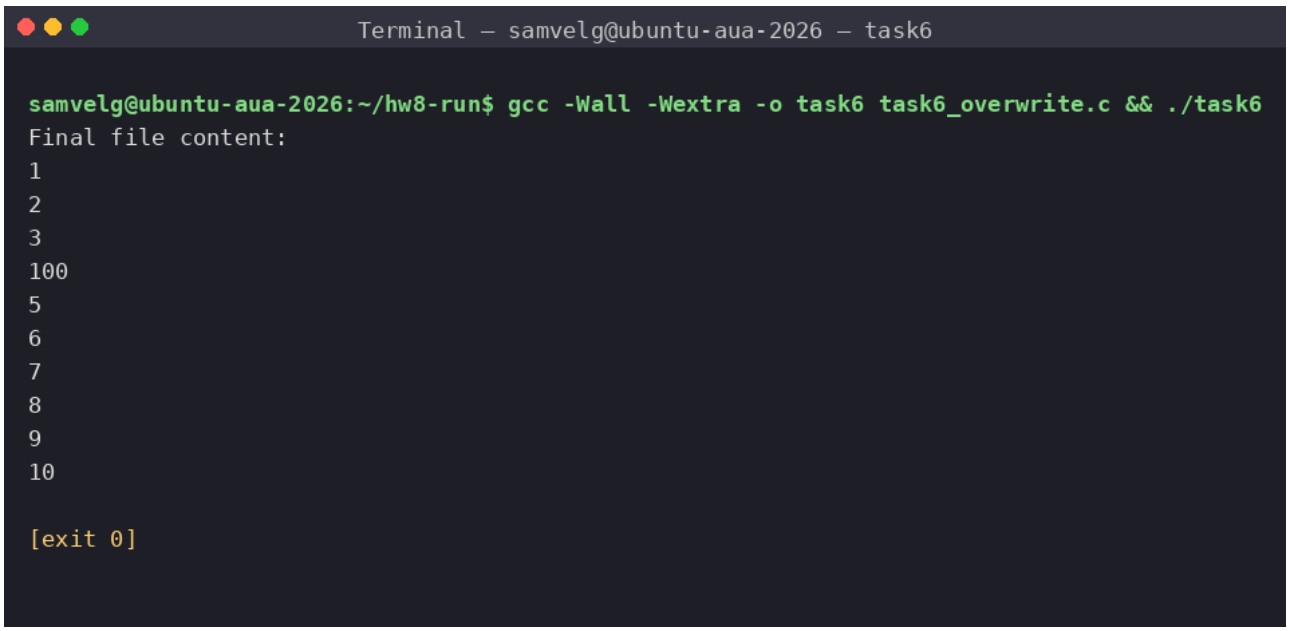
To check that it was really sparse I ran `ls -l`, which also showed 1048584, and `du -h`, which showed only 8.0K. The difference is there because the 1 MiB gap is a hole. The filesystem doesn't store blocks of zeros for it, so only the blocks with real data take up space on disk.



Task 6: Overwriting a line in place

This program first writes the numbers 1 to 10, one per line, into `numbers.txt`. Then it reopens the file with `O_RDWR` and reads it one byte at a time, counting newlines until it has seen three. That tells me the byte offset where line 4 starts. I skip past the old `"4\n"` and read the rest of the file into a buffer that I grow with `realloc()`.

Since `"100\n"` is longer than `"4\n"`, I can't just write over it, or it would eat into line 5. So I `lseek()` back to the start of line 4, write `"100\n"`, write the saved remainder after it, and then call `ftruncate()` at the current offset in case the file ever gets shorter. When I printed the file afterwards it read 1, 2, 3, 100, 5, ..., 10.



Task 7: Comparing two files byte by byte

The last program asks for two paths, opens both read-only, and reads them in 4096-byte chunks side by side. Inside each chunk I compare byte by byte, keeping a running index. At the first mismatch it prints `Files differ at byte N` (counting from 0) and exits with status 1. If one file runs out before the other and everything so far matched, the first difference is at the point where the shorter file ends. If both reach EOF together with no mismatch, it prints `Files are identical` and returns 0.

I tried three cases. Two files that differ at index 2 gave "Files differ at byte 2". Two identical files gave "Files are identical". A file that is a shorter prefix of the other gave "Files differ at byte 4".

